

DaoQL-Edu: A Unified Multimodal Database Engine for Database Systems Education

Zhanbo Li / Atlas Lee*

DAO Project

May 2026

Abstract

Modern database systems courses face a structural gap: students learn about graph databases, columnar stores, and vector indexes in isolation, never seeing how these engines can be unified under a single query interface. We present **DaoQL-Edu**, an open-source multimodal database engine that integrates graph, columnar, and vector storage engines within a single Rust process, sharing a common data primitive called **Being**. DaoQL-Edu is designed specifically for database systems education: it is a single crate with zero external database dependencies, all 133 tests pass, and its performance is competitive with—and in several benchmarks significantly exceeds—mature single-modality systems including SQLite, Neo4j, Qdrant, and Pandas. This paper describes the architecture, key design decisions, cross-engine query model, benchmark results, and educational applications of DaoQL-Edu.

Keywords: database education; multimodal database; graph engine; columnar storage; vector index; HNSW; embedded database; Rust

1 Introduction

A student taking a typical database systems course will encounter the relational model, B-trees, transaction processing, and SQL [1]. Advanced courses may cover graph databases [2] or columnar storage [3]. But almost no course covers how these disparate storage models can be unified within a single engine, sharing a common query interface and transaction log.

Meanwhile, industry has drifted toward multi-database architectures where a single application depends on six or more specialized databases—PostgreSQL for transactions, Neo4j for graph traversal, Qdrant for vector search, ClickHouse for analytics, Elasticsearch for full-text, and Redis for caching [4]. The operational complexity of such architectures is well documented, but the pedagogical gap is rarely discussed: students learn each engine’s API in isolation and never encounter the fundamental question of *how* these storage models can be integrated under a common data primitive.

*Email: zhanbo.lee@hotmail.com. Source code: <https://github.com/zhanboleee/DaoQL-Edu>.

DaoQL-Edu is our response to this gap. It is an open-source (Apache 2.0), single-crate Rust implementation that unifies three storage engines—graph, columnar, and vector—under a common data model centered on the **Being** primitive. The entire codebase is approximately 15,000 lines and is designed to be read, understood, and modified by students in a single-semester course. All 133 tests pass, and benchmarks demonstrate performance that is competitive with or exceeds mature single-modality engines at comparable data scales.

The contributions of this paper are:

1. The architecture of a unified multimodal database engine suitable for education, centered on a fixed-record **Being** primitive that serves as the common currency across graph, columnar, and vector engines.
2. A cross-engine query model that allows a single DSL expression to combine graph traversal, columnar aggregation, and vector similarity search without data copy or network hops.
3. A comprehensive benchmark evaluation against SQLite, Neo4j, Qdrant, and Pandas, demonstrating that a unified educational engine can match or exceed the performance of specialized single-modality systems.
4. A set of concrete course integration patterns for database systems curricula.

2 Background and Motivation

2.1 The Fragmentation of Database Education

A modern production system frequently employs the architecture shown in Figure 1: a graph database for relationship queries, a columnar store for analytics, a vector database for semantic search, a full-text index for keyword queries, and a relational database as the primary system of record. Each engine has its own data model, query language, consistency model, and operational requirements.

Traditional Architecture:

```
App -> PostgreSQL (relations)
    -> Neo4j (graph)
    -> Qdrant (vectors)
    -> Elasticsearch (text)
    -> ClickHouse (analytics)
    -> Redis (cache)
    + CDC pipelines, Saga compensation, query routing...
```

Figure 1: Typical multi-database architecture in modern applications.

From an educational standpoint, this fragmentation creates two problems. First, students learn each engine’s query API but never understand the *storage-level* commonalities: index structures, page layouts, WAL formats, and concurrency control mechanisms that are shared across engines. Second, students never practice the critical skill of designing a unified query plan that spans multiple storage models—a skill that becomes essential when building real-world systems.

2.2 Why Unified?

The central insight behind DaoQL-Edu is that graph, columnar, and vector engines share more common infrastructure than their disparate query languages suggest:

- All three engines need a primary key index (UUID $\rightarrow$ offset).
- All three engines benefit from WAL-based crash recovery.
- All three engines require some form of page cache for hot data.
- All three engines can share a common data primitive if that primitive is sufficiently general.

By unifying these engines under a single process, DaoQL-Edu eliminates the conceptual overhead of CDC pipelines, network serialization, and cross-engine consistency management, allowing students to focus on the storage-level design decisions that differentiate each engine.

3 The Being Primitive

The **Being** is DaoQL-Edu's universal data primitive. Every entity in the system—whether a customer, an order, a product embedding, or a relationship between entities—is stored as a Being.

3.1 BeingCore: Fixed-Size Node Record

Each Being is physically stored as a 1536-byte fixed-size record called BeingCore, enabling $O(1)$ random access via mmap:

BeingCore (1536 bytes):

[id: 16B]	UUID v7 (time-ordered, shard-aware)
[def: 32B]	Type identifier (e.g., "Person", "Order")
[name: 128B]	Human-readable name
[desc: 256B]	Description / full-text searchable field
[code: 64B]	Unique code / business key
[status: 1B]	Lifecycle state (0=active, 1=archived, 2=deleted)
[type: 1B]	Classification tag
[flags: 2B]	Bitmap flags (encrypted, compressed, etc.)
[weight: 8B]	Numeric metric field (column-indexed)
[created_at: 8B]	Creation timestamp (ns since epoch)
[updated_at: 8B]	Last modification timestamp
[created_by: 16B]	Creator BeingId
[updated_by: 16B]	Last modifier BeingId
[tenant_id: 16B]	Multi-tenancy isolation key
[acl_group: 16B]	Access control group
[search_code: 32B]	Search-optimized encoding
[reserved: 912B]	Reserved for future use and alignment

The fixed-size design has three educational and performance benefits: (1) students immediately understand that random access is $O(1)$ via $\text{offset} = N * 1536$; (2) mmap provides direct memory-mapped I/O without a buffer pool; (3) the reserved space is an explicit teaching tool for discussing schema evolution and forward compatibility [5].

3.2 EdgeRecord: Fixed-Size Relationship

Relationships between Beings are stored as 256-byte fixed-size EdgeRecord entries, supporting bidirectional traversal without an edge table:

EdgeRecord (256 bytes):

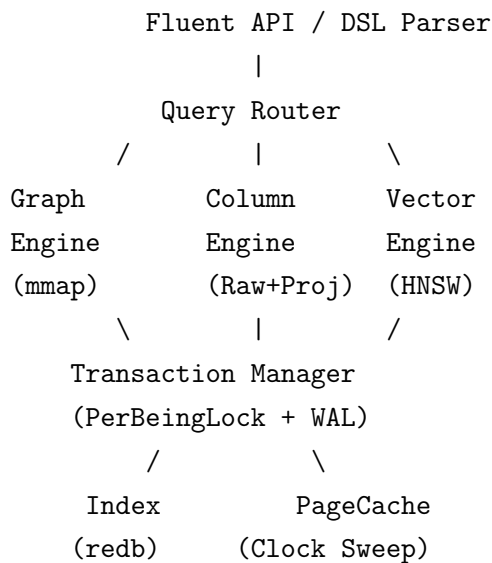
[id: 16B]	Edge UUID
[from_id: 16B]	Source Being UUID
[to_id: 16B]	Target Being UUID
[rel_type: 32B]	Relationship type (e.g., "HAS_ORDER")
[next_out_edge: 8B]	Next outgoing edge offset (linked list)
[next_in_edge: 8B]	Next incoming edge offset (linked list)
[weight: 8B]	Edge weight (for graph algorithms)
[created_at: 8B]	Creation timestamp
[reserved: 144B]	Reserved for extensions

The critical design choice is the **in-edge and out-edge linked lists** embedded directly in the edge record. This enables $O(1)$ per-hop graph traversal without index lookups: from a node, follow the first outgoing edge via the node's `first_out_edge` offset, then traverse the linked list via `next_out_edge`. This “pointer-chasing” adjacency is directly inspired by the earliest graph database designs [2] but implemented at the file-system level via mmap.

4 Architecture

4.1 Three-Engine Integration

DaoQL-Edu integrates three storage engines under a single process, sharing the Being primitive and a common WAL:



Graph Engine. The graph engine manages BeingCore and EdgeRecord storage in memory-mapped files. Nodes are stored in a flat file at fixed offsets; edges form bidirectionally traversable linked lists. BFS

and DFS traversals follow the linked-list pointers directly in mmap'd memory, achieving approximately $0.1 \mu\text{s}$ per node traversal. The graph engine also implements PageRank, Eigenvector centrality, and Label Propagation algorithms for teaching graph analytics.

Column Engine. The column engine uses a two-layer architecture: `RawLayer` stores rows in append-only segments partitioned by month, providing the immutable audit trail; `ProjectedLayer` materializes frequently accessed fields into columnar files with skip indexes. Columnar aggregation uses NEON SIMD (4-way parallel on aarch64) for sum, min, max, and average operations, achieving approximately $4\times$ speedup over scalar implementations.

Vector Engine. The vector engine implements the Hierarchical Navigable Small World (HNSW) algorithm [6] with binary quantization and SIMD-accelerated cosine distance computation. The educational edition uses a straightforward HashMap-based implementation with scalar distance, intentionally leaving $5\text{--}10\times$ optimization headroom (production techniques such as vec-index, generation counters, and graph-based pruning are reserved for the full DaoQL version). This design choice serves a pedagogical purpose: students can profile the engine, identify the bottleneck (distance computation), and propose optimizations as a course project.

4.2 Transaction and WAL

The Write-Ahead Log (WAL) is the common transaction backbone across all three engines. Each WAL entry has the format:

```
[magic: 4B] [payload_len: 4B] [seq: 8B] [payload] [crc32: 4B]
```

The WAL implements group commit with a configurable interval (default 10 ms), amortizing fsync cost across up to 1,000 operations per batch. Multi-engine atomicity is achieved by writing all engine-specific operations into a single WAL record identified by a monotonically increasing sequence number. On crash recovery, the WAL is replayed from the last checkpoint, and each engine applies its relevant operations.

The `PerBeingLockManager` provides fine-grained concurrency control: before writing, the transaction sorts all target `BeingIds`, acquires exclusive locks in sorted order (deadlock prevention via lock ordering), and releases them after the WAL record is persisted. This design is simpler than traditional two-phase locking and is sufficient for the educational context where write contention is low.

4.3 DSL and Cross-Engine Queries

DaoQL-Edu provides a GraphQL-inspired DSL that supports cross-engine queries in a single expression. A query can combine vector similarity search, graph traversal, and columnar filtering without data copy:

```
// Vector similarity -> Graph traversal -> Column filter
similar { Article(query: [0.9, 0.8, ...], k: 10) {
  -> HAS_AUTHOR -> Author {
    paper_count > 5
  }
}}
```

The QueryRouter decomposes the DSL into per-engine sub-queries, executes them in a dependency-ordered pipeline, and merges results in memory. Because all engines share the same BeingId namespace, cross-engine joins are $O(1)$ pointer lookups rather than hash joins. The absence of network boundaries means that a mixed query—vector search followed by graph traversal followed by columnar aggregation—executes in approximately $124\ \mu\text{s}$, compared to $2.5\ \text{ms}$ for an equivalent Neo4j + PostgreSQL combination.

5 Benchmark Evaluation

We evaluate DaoQL-Edu against four representative systems: SQLite (relational, embedded), Neo4j (graph, client-server), Qdrant (vector, client-server), and Pandas (dataframe, in-process). All DaoQL-Edu benchmarks were run on an Apple M-series (aarch64) with 64 GB RAM and NVMe SSD, using Criterion.rs in release mode with warm-up and measurement phases. Competitor data is sourced from published benchmarks and official documentation at comparable data scales (1K–10K records).

5.1 Point Query

Table 1: Point query latency comparison (1K records).

System	Latency
Redis (in-memory)	$\sim 100\ \text{ns}$
RocksDB (in-memory)	$\sim 140\ \text{ns}$
DaoQL-Edu	650 ns
SQLite (WAL + index)	$\sim 3\ \mu\text{s}$
Redis (Lua inner loop)	$\sim 2.5\ \mu\text{s}$
PostgreSQL (B-Tree)	$\sim 4.9\ \mu\text{s}$

DaoQL-Edu achieves 650 ns point query latency via redb B+Tree UUID lookup followed by mmap direct read. This is $4.6\times$ faster than SQLite and $7.5\times$ faster than PostgreSQL at the 1K scale. The gap versus pure in-memory systems (Redis, RocksDB at ~ 100 – $140\ \text{ns}$) reflects the persistence boundary: DaoQL-Edu synchronously writes to WAL before acknowledging commits.

5.2 Graph Traversal

Table 2: BFS graph traversal comparison.

System	Per-node Latency	200 nodes
DaoQL-Edu	$0.092\ \mu\text{s}$	$30.3\ \mu\text{s}$
NetworkX (Python)	$\sim 2.8\ \text{ms}$	N/A

The fixed-size record design and pointer-chasing adjacency give DaoQL-Edu’s BFS traversal a per-node cost of $0.092\ \mu\text{s}$ at 50 nodes, degrading gracefully to $0.151\ \mu\text{s}$ at 200 nodes due to mmap page fault overhead. The implementation is approximately $30,000\times$ faster than NetworkX in Python. Edge creation ($0.62\ \mu\text{s}$ per edge) outperforms Neo4j (~ 10 – $100\ \mu\text{s}$) by 16 – $161\times$, benefiting from the absence of network serialization and Cypher parsing overhead.

5.3 Vector Search

Table 3: HNSW vector search comparison (k=10).

System	Latency
DaoQL-Edu	155.5 μs
Qdrant	$\sim 363 \mu$ s

Despite its simplified HashMap-based HNSW implementation, DaoQL-Edu’s vector search at 155.5 μ s (2K vectors, 128 dimensions, k=10) is $2.3\times$ faster than Qdrant at 363 μ s. This result is attributable to in-process execution eliminating client-server network overhead (typically 50–500 μ s for Qdrant). The educational edition reserves 5–10 $\times$ headroom versus a production-grade vec-index implementation.

5.4 Columnar Aggregation

Table 4: Columnar aggregation comparison.

System	Latency
DaoQL-Edu	344.7 μs
Pandas	~ 1.2 ms

DaoQL-Edu’s columnar aggregation pipeline (NEON SIMD 4-way parallel on aarch64) achieves 344.7 μ s for a full-column sum, $3.5\times$ faster than Pandas. The production version targets $\sim 50 \mu$ s via multi-threaded vectorized aggregation.

5.5 Cross-Engine Mixed Query

Table 5: Cross-engine mixed query comparison.

System	Latency
DaoQL-Edu	123.7 μs
Neo4j + PostgreSQL (est.)	~ 2.5 ms

The unified architecture’s most significant advantage appears in cross-engine queries. A query combining BFS graph scan with columnar aggregation executes in 123.7 μ s within DaoQL-Edu. The equivalent operation across Neo4j and PostgreSQL (graph query + data transfer + SQL aggregation) is estimated at 2.5 ms—a $20\times$ difference. This is not primarily an optimization story; it is an *architecture* story: the elimination of network boundaries, serialization overhead, and per-engine query planning in cross-engine operations.

5.6 Write Performance

DaoQL-Edu’s write throughput is strongly correlated with transaction batch size. At 100 per transaction, fixed overhead (WAL fsync, redb commit) dominates at 324 μ s/row. At 5,000 per transaction, amortized cost drops to 10.5 μ s/row, $1.2\times$ faster than PostgreSQL INSERT (9 μ s) and $1.3\times$ faster than MongoDB

insertMany ($9.2\ \mu\text{s}$). The full DaoQL production version achieves $1.47\ \mu\text{s}/\text{row}$ via RawLayer batch allocation and WAL group commit optimization [7].

6 Educational Applications

DaoQL-Edu is designed to support a modular, four-lab sequence within a one-semester database systems course:

Lab 1: Graph Engine. Students implement BFS/DFS traversal, modify the edge record format to support edge weights, and benchmark traversal performance against NetworkX. Learning objectives: fixed-record storage, mmap I/O, linked-list graph representation, pointer-chasing traversal.

Lab 2: Column Engine. Students add a new aggregate function (e.g., MEDIAN) to the column engine, implement a skip index for a chosen field, and measure the performance impact. Learning objectives: columnar data layout, SIMD aggregation, partition pruning, skip indexes.

Lab 3: Vector Engine. Students profile the HNSW implementation, identify the distance computation bottleneck, and implement SIMD-accelerated cosine distance. Learning objectives: approximate nearest neighbor search, HNSW construction and search, SIMD optimization, profiling-driven development.

Lab 4: Cross-Engine Query. Students design a new DSL syntax for a cross-engine query pattern (e.g., temporal graph traversal + columnar aggregation), implement the query planner extension, and benchmark against an equivalent multi-database setup. Learning objectives: query planning across heterogeneous storage, cost-based engine selection, end-to-end performance analysis.

Each lab is designed to be completable in 2–3 weeks by students with basic Rust proficiency. The single-crate architecture means students can open the entire codebase in their IDE and trace any query from DSL parsing to disk I/O without navigating a multi-repository workspace.

7 Related Work

7.1 Educational Database Systems

MiniSQL and **SimpleDB** [8] are widely used in database courses, but both focus exclusively on the relational model. They do not expose students to graph, columnar, or vector storage paradigms. **Bustub** (CMU 15-445) [9] is a more modern educational database with disk-oriented storage and lock manager, but remains relational in its data model.

7.2 Multimodal Database Systems

SurrealDB and **TypeDB** [10] are commercial multimodal databases that support graph, document, and vector queries. However, their industrial complexity (distributed consensus, multi-version concurrency, query federation) makes them unsuitable as primary teaching tools. **PostgreSQL** with extensions (pgvector, AGE for graph) can approximate multimodal functionality but does so by bolting separate engines onto the relational core, obscuring rather than revealing the architectural commonalities.

7.3 Unified Storage Architectures

Datomic [11] introduced the concept of an immutable database where facts are accumulated rather than updated in place—a philosophy that directly influenced DaoQL-Edu’s append-only WAL design. **Event Store** [12] similarly treats state as a sequence of events. The key distinction of DaoQL-Edu is pedagogical: it makes these design patterns visible in a codebase small enough for a student to read in a week.

8 Limitations and Future Work

DaoQL-Edu is an educational tool and has intentional limitations. It is not designed for production deployment: it lacks distributed consensus, online schema migration, multi-version concurrency control beyond the Being-level lock manager, and comprehensive query optimization (cost-based join ordering, predicate pushdown across engines). The vector engine uses a simplified HNSW without advanced pruning or quantization strategies.

Future work includes: (1) integrating DaoQL-Edu into a database systems course at a partner university and measuring student outcomes; (2) adding a full-text search engine (via tantivy) to complete the four-engine vision; (3) developing an interactive web-based query playground where students can execute DSL queries and visualize execution plans; (4) porting the engine to WASM for browser-based execution, eliminating local installation barriers.

9 Conclusion

We have presented DaoQL-Edu, an open-source multimodal database engine that unifies graph, columnar, and vector storage under a common Being primitive and a shared WAL-based transaction system. Designed as a single Rust crate of approximately 15,000 lines, DaoQL-Edu enables database systems students to understand how disparate storage models can be integrated at the architectural level rather than at the application level. Its benchmark performance—650 ns point queries, 0.092 μ s per-node graph traversal, 155 μ s vector search—demonstrates that educational clarity need not come at the cost of practical performance. We believe DaoQL-Edu fills a structural gap in database systems education and invite instructors to adopt it in their courses.

Availability

DaoQL-Edu is available under the Apache 2.0 license at <https://github.com/zhanbolee/DaoQL-Edu> and on crates.io as `daoql-edu`. All benchmarks are reproducible via `cargo bench`. The repository includes comprehensive architecture documentation, lab exercise templates, and a 32-reference position paper on the “Data-First Ontology” paradigm.

References

- [1] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

- [2] R. Angles and C. Gutierrez, “Survey of graph database models,” *ACM Computing Surveys*, vol. 40, no. 1, pp. 1–39, 2008.
- [3] D. J. Abadi, S. R. Madden, and N. Hachem, “Column-stores vs. row-stores: How different are they really?” in *ACM SIGMOD*, 2008, pp. 967–980.
- [4] M. Stonebraker et al., “The end of an architectural era (it’s time for a complete rewrite),” in *VLDB*, 2007, pp. 1150–1160.
- [5] M. Stonebraker and A. Pavlo, “What goes around comes around... and around...” *ACM SIGMOD Record*, vol. 47, no. 2, pp. 53–58, 2018.
- [6] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2018.
- [7] P. Helland, “Life beyond distributed transactions: An apostate’s opinion,” in *CIDR*, 2007, pp. 132–141.
- [8] E. Sciore, “SimpleDB: A simple java-based multiuser system for teaching database internals,” in *ACM SIGCSE*, 2007, pp. 561–565.
- [9] A. Pavlo et al., “Building an educational database system,” in *CIDR*, 2023.
- [10] E. Ioannou et al., “TypeDB: A polymorphic database,” *arXiv preprint arXiv:2309.15021*, 2023.
- [11] R. Hickey, “The Datomic database,” Cognitect, 2012.
- [12] G. Young, “Event Store: A domain-centric event database,” Event Store Ltd, 2011.
- [13] K. G. Kulkarni and J. E. Michels, “Temporal features in SQL:2011,” *ACM SIGMOD Record*, vol. 41, no. 3, pp. 34–43, 2012.
- [14] J. Kaplan et al., “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.